

LECTURE – 15

BREADTH FIRST SEARCH OF A GRAPH

Simple, basic algorithm. Finds “the shortest path”, in terms of number of edges, from source vertex s to every other vertex in G . Works on both directed and undirected graphs.

Meaning of “breadth first”:

This algorithm discovers all vertices at distance k from s , before discovering any vertex at distance $k+1$.

BFS (G, s)

```
// Initialization
for each vertex  $u$  in  $V[G] - \{s\}$ 
    color( $u$ ) = WHITE;
    dist[ $u$ ] = INF;
    pred[ $u$ ] = NIL;

color[ $s$ ] = GRAY;
dist[ $s$ ] = 0;
pred[ $s$ ] = NIL;
 $Q$  = NULL-SET;
ENQUEUE( $Q, s$ );

// Now the search starts
while  $Q \neq \text{NULL-SET}$ 
    // Take next node  $u$ 
     $u$  = DEQUEUE( $Q$ );
    for each  $v$  in ADJ[ $u$ ];
        if color[ $v$ ] = WHITE
            color[ $v$ ] = GRAY;
            dist[ $v$ ] = dist[ $u$ ]+1;
            pred[ $v$ ] =  $u$ ;
            ENQUEUE( $Q, v$ );
    // We are done with node  $u$ 
    color[ $u$ ] = BLACK;
```

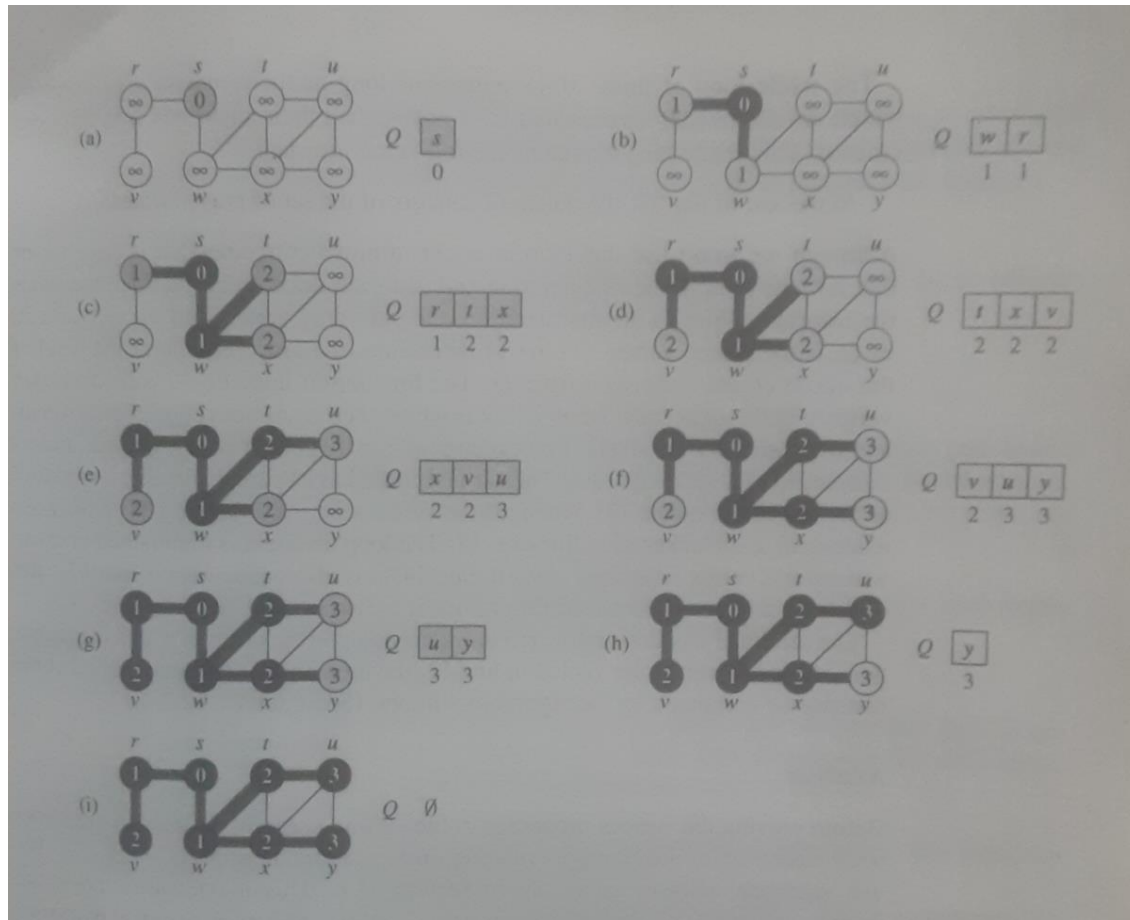
Loop invariant of **while** loop:

At the time of the **while** test, the queue Q consists of the set of gray vertices.

Running time analysis:

Each vertex is queued and de-queued (at most) once, each of which operations is $O(1)$. Later, each edge is examined at most once, so the second part of the algorithm is $O(E)$. Hence the total running time is $O(V+E)$.

Example:



Provable properties of BFS

Notation: The **shortest path distance** $\delta(s,v)$ from source vertex s to any other vertex v is the minimum number of edges in any path from s to v .

1. For any edge (u,v) in E , $\delta(s,v) \leq \delta(s,u)+1$.

2. After BFS is run, every vertex v in V that is reachable from s is discovered, and $\text{dist}[v] = \delta(s,v)$.

3. The predecessor relationship discovered by BFS defines a breadth first tree of graph G with source vertex s .

Function **`print-path( $G, s, v$ )`** $\rightarrow$ self- study.